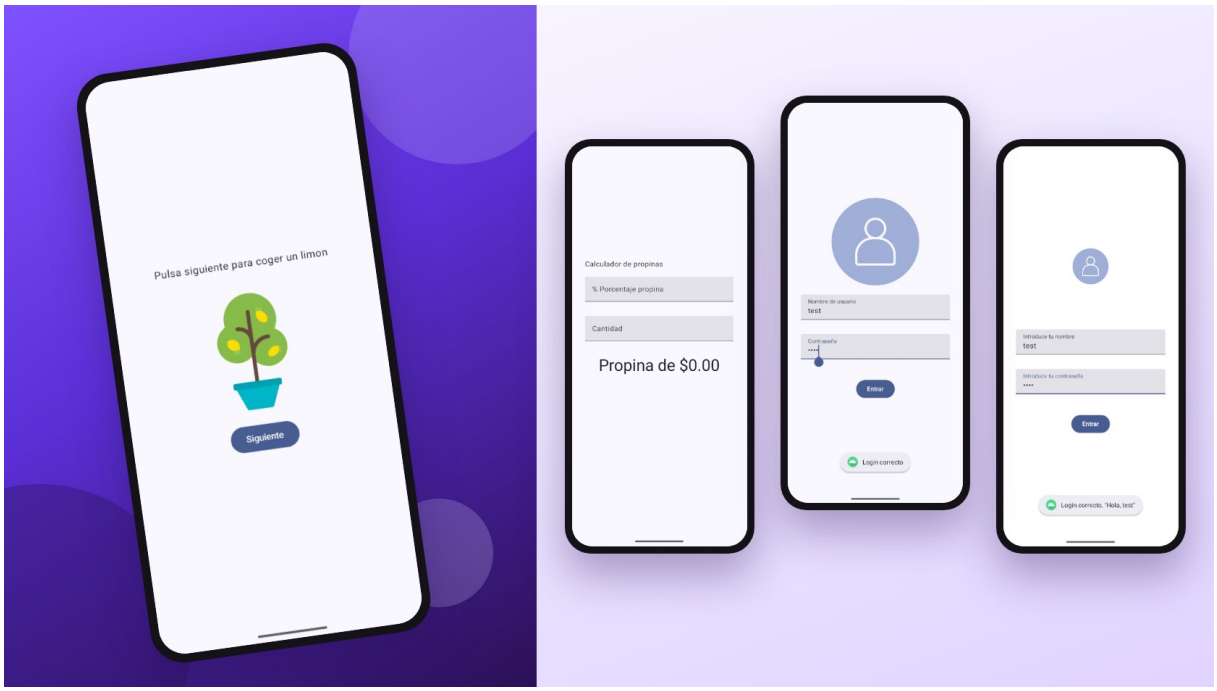


Repositorio Interactivo

Eva Gallardo Romero

13 de febrero de 2026

Índice

Enlace al repositorio de GitHub **2**

1. Descripción general **2**

2. Diagrama de clases **2**

3. Diseño **2**
3.1. Menú principal 2
3.2. Calculadora de propinas 2
3.3. Limonada 4
3.4. Login 4
3.5. Galería 4
3.6. Test de 10 preguntas 4
3.7. Login con ViewModel 4

4. Requisitos del sistema **6**
4.1. Requisitos funcionales 6
4.2. Requisitos no funcionales 6

5. Documentación técnica **6**
5.1. Proyecto.kt 6
5.2. MisProyectos.kt 7
5.3. Home.kt 7
5.4. Propinas.kt 9
5.5. Botones.kt 10
5.6. Login.kt 11
5.7. Miniaturas.kt 12
5.8. MisMiniaturas.kt 12
5.9. Galeria.kt 13
5.10. Pregunta.kt 14
5.11. MisPreguntas.kt 14
5.12. Score.kt 15
5.13. Formulario.kt 15
5.14. UIState.kt 16
5.15. MainViewModel.kt 17
5.16. MainActivity.kt 17
5.17. Theme.kt, Color.kt y Type.kt 18

6. Mejoras futuras **19**

Enlace al repositorio de GitHub

Puedes consultar el proyecto completo y descargable en el siguiente enlace: RepositorioInteractivo.

La aplicación instalable (APK) está en la sección Releases.

1. Descripción general

Repositorio Interactivo es una aplicación para Android hecha con **Kotlin** y **Jetpack Compose** que reúne **seis mini-proyectos** en una sola app. La pantalla principal muestra un **carrusel** con una tarjeta por mini-proyecto y cada uno se abre en su propia `Activity`:

1. **Calculadora de propinas**: calcula la propina de una cantidad con el porcentaje que se escriba.
2. **Limonada**: cuatro pasos que cambian de imagen y de texto al pulsar un botón.
3. **Login**: formulario de usuario y contraseña que avisa si falta algún campo.
4. **Galería**: lista de tarjetas con imagen y nombre cargadas desde los recursos.
5. **Test**: diez preguntas con respuestas de distintos tipos gracias a una clase genérica `Pregunta<T>`.
6. **Login con ViewModel**: el mismo login, pero con el estado guardado en un `ViewModel` y expuesto con `StateFlow`.

La hice para practicar la interfaz declarativa de Compose: funciones `@Composable`, el **estado** con `remember` y `mutableStateOf`, las listas perezosas (`LazyRow` y `LazyColumn`), los componentes de **Material 3**, las clases genéricas de Kotlin y, al final, la arquitectura recomendada por Google con **ViewModel** y **StateFlow**. La app no necesita internet ni guarda datos.

2. Diagrama de clases

`Home` es la `Activity` de entrada. Lee la lista de `MisProyectos` y abre cada mini-proyecto con un `Intent`. Los mini-proyectos que necesitan datos tienen sus propias clases de modelo: la galería, sus miniaturas; el test, sus preguntas y la puntuación; y el login avanzado, su `ViewModel` con el estado.

3. Diseño

Todas las pantallas usan el tema de **Material 3** de la app (`ProyectitosInteractivosTheme`) sobre fondo claro, con los campos de texto rellenos y los botones redondeados por defecto.

3.1. Menú principal

Al abrir la app se ve el título *«Repositorio interactivo»* y un **carrusel horizontal**. Cada tarjeta enseña una captura del mini-proyecto, su nombre, una descripción corta y el botón **Acceder**, gris oscuro, que lo abre. Las capturas de las tarjetas son las mismas que aparecen en los apartados siguientes.

3.2. Calculadora de propinas

Dos campos con **teclado numérico**: el porcentaje de propina y la cantidad. Debajo, la propina se actualiza mientras se escribe y se muestra con el formato de moneda del móvil. Si el porcentaje se deja vacío, se usa un 15 %.

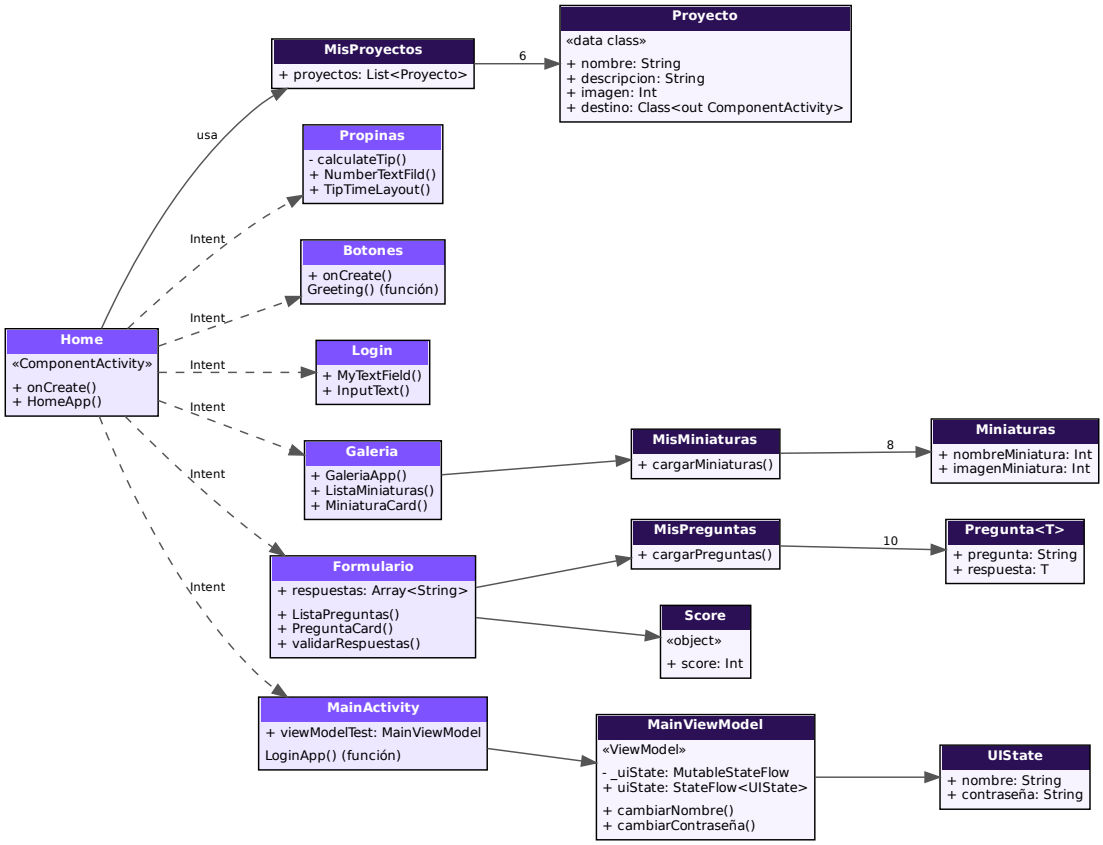


Figura 1: Diagrama de clases de Repositorio Interactivo

3.3. Limonada

Un texto, una imagen y el botón **Siguiente**. Cada pulsación avanza un paso: coger un limón del árbol, exprimirlo, beberse la limonada y volver a empezar.

3.4. Login

Una imagen de perfil, el campo del usuario, el de la contraseña (con los caracteres ocultos) y el botón **Entrar**. Si los dos campos tienen texto aparece *«Login correcto»*; si no, *«Rellena todos los campos»*.

3.5. Galería

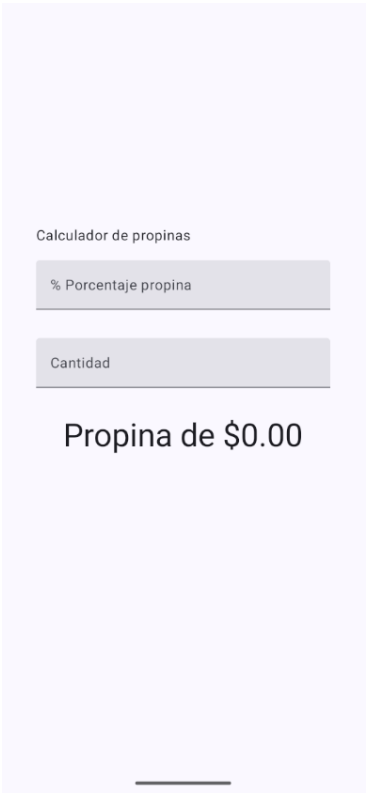
Una lista vertical de tarjetas. Cada una tiene una imagen que ocupa todo el ancho y su nombre debajo.

3.6. Test de 10 preguntas

Diez tarjetas con una pregunta y un campo para escribir la respuesta, y el botón **Finalizar Test** fijo abajo, que enseña *«Has acertado N preguntas»*.

3.7. Login con ViewModel

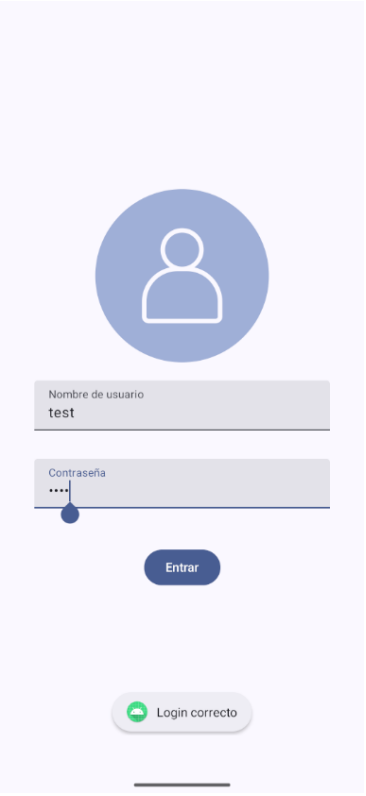
Parecido al login anterior, con la imagen más pequeña. Al entrar saluda por el nombre: *«Login correcto, "Hola, test"»*.



3.2. Propinas



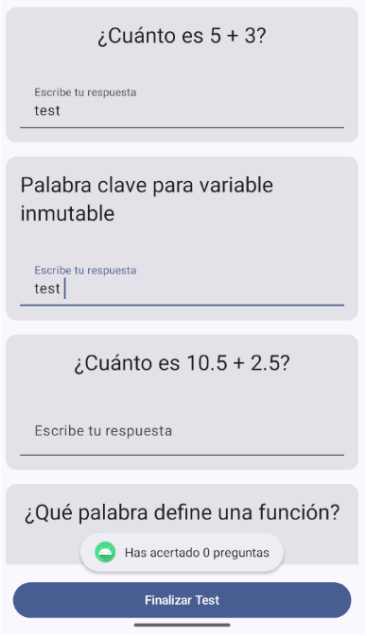
3.3. Limonada



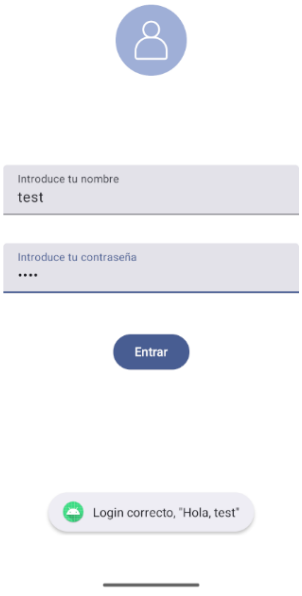
3.4. Login



3.5. Galería



3.6. Test



3.7. Login con ViewModel

Figura 2: Capturas de los mini-proyectos

4. Requisitos del sistema

4.1. Requisitos funcionales

ID	Requisito funcional
RF01	La aplicación debe mostrar los mini-proyectos en un **carrusel** con imagen, nombre y descripción.
RF02	El botón **Acceder** debe abrir la pantalla del mini-proyecto elegido.
RF03	La calculadora debe **calcular la propina** con la cantidad y el porcentaje escritos.
RF04	Si no se escribe porcentaje, la calculadora debe usar un **15 %**.
RF05	La propina debe mostrarse con el **formato de moneda** del dispositivo.
RF06	La limonada debe **avanzar un paso** en cada pulsación y volver al primero después del cuarto.
RF07	El login debe **ocultar la contraseña** mientras se escribe.
RF08	El login debe avisar con un mensaje si **falta algún campo** y confirmar el acceso si están los dos.
RF09	La galería debe mostrar una **lista de tarjetas** con imagen y nombre.
RF10	El test debe mostrar **10 preguntas** con un campo de respuesta cada una.
RF11	Al pulsar **Finalizar Test** se debe mostrar el número de aciertos.
RF12	El login con ViewModel debe **saludar por el nombre** escrito al entrar.

4.2. Requisitos no funcionales

ID	Requisito no funcional
RNF01	La interfaz debe estar hecha con **Jetpack Compose** y **Material 3**.
RNF02	Cada mini-proyecto debe ser una **Activity independiente**, de forma que añadir uno nuevo solo requiera añadirlo a `MisProyectos`.
RNF03	El estado del login avanzado debe estar en un **ViewModel** y exponerse con **StateFlow**.
RNF04	Las imágenes y los textos de la galería deben cargarse desde los **recursos** de la app.
RNF05	La aplicación debe funcionar **sin conexión a internet**.
RNF06	La aplicación debe funcionar en **Android 7.0 (API 24)** o superior.

5. Documentación técnica

Este anexo agrupa la documentación por archivos, separándola en la **ubicación** donde se encuentra, una **descripción general** y las **funciones** explicadas de manera individual con su **código** adjunto y comentado. Se omiten los `import` y las funciones `@Preview`, que solo sirven para ver el diseño en Android Studio.

5.1. Proyecto.kt

5.1.1. Ubicación

`com.evagr.proyectitosinteractivos.data`

Descripción general:
La data class `Proyecto` describe una tarjeta del menú: el nombre, la descripción, la imagen y la `Activity` que se abre al pulsarla.

5.1.2. Funciones

Proyecto **Descripción:** Declara los cuatro atributos en el constructor principal. El destino es cualquier clase que herede de `ComponentActivity`.

Código comentado:

```
data class Proyecto(
    val nombre: String,                          // título de la tarjeta
    val descripcion: String,                     // texto corto debajo del título
    val imagen: Int,                             // id del drawable con la captura
    val destino: Class<out ComponentActivity>    // pantalla que se abre
)
```

5.2. MisProyectos.kt

5.2.1. Ubicación

`com.evagr.proyectitosinteractivos.data`

Descripción general:
La clase `MisProyectos` guarda la lista de los seis mini-proyectos que se enseñan en el carrusel. Para añadir uno nuevo basta con añadir otro `Proyecto` a la lista.

5.2.2. Funciones

proyectos **Descripción:** Lista inmutable con un `Proyecto` por mini-proyecto (se muestran dos; los otros cuatro siguen el mismo patrón).

Código comentado:

```
class MisProyectos {
    val proyectos = listOf(
        Proyecto(
            "Propinas",
            "Calculadora de propinas",
            R.drawable.propinas,
            Propinas::class.java      // la Activity se pasa como clase
        ),
        Proyecto(
            "Botones",
            "Interacción con botones",
            R.drawable.btn_dinamicos,
            Botones::class.java
        ),
        // ... Login, Galería, Formulario y Login avanzado (MainActivity)
    )
}
```

5.3. Home.kt

5.3.1. Ubicación

`com.evagr.proyectitosinteractivos.ui.theme`

Descripción general:
La clase `Home` es la `Activity` de entrada de la app (la que tiene el `LAUNCHER` en el `AndroidManifest.xml`). Dibuja el título y el carrusel de mini-proyectos.

5.3.2. Funciones

onCreate **Descripción:** Activa el modo de pantalla completa (*edge to edge*) y carga `HomeApp` dentro del tema de la app.

Código comentado:

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    enableEdgeToEdge()
    setContent {
        ProyectitosInteractivosTheme {
            Surface(
                modifier = Modifier.fillMaxSize(),
                color = colorResource(R.color.white)
            ) {
                HomeApp()
            }
        }
    }
}
```

HomeApp **Descripción:** Recorre la lista de `MisProyectos` con un `LazyRow` y dibuja una tarjeta por proyecto. El botón **Acceder** abre su `Activity` con un `Intent`.

Código comentado:

```
@Composable
fun HomeApp() {
    val context = LocalContext.current          // hace falta para lanzar el Intent
    val proyectos = MisProyectos().proyectos

    Column(
        modifier = Modifier.fillMaxSize().background(colorResource(R.color.white)),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Spacer(modifier = Modifier.height(20.dp))
        Text(
            text = "Repositorio interactivo",
            fontSize = 24.sp,
            fontWeight = FontWeight.Bold,
            modifier = Modifier.fillMaxWidth().padding(top = 20.dp),
            color = Color.Black,
            textAlign = TextAlign.Center
        )
        Spacer(modifier = Modifier.height(20.dp))

        // Carrusel horizontal: solo se componen las tarjetas visibles
        LazyRow(
            modifier = Modifier.fillMaxWidth(),
            contentPadding = PaddingValues(horizontal = 16.dp),
            horizontalArrangement = Arrangement.spacedBy(16.dp)
        ) {
            items(proyectos) { proyecto ->
                Column(
                    modifier = Modifier.fillMaxWidth(),
                    horizontalAlignment = Alignment.CenterHorizontally
                ) {
                    Card(modifier = Modifier.size(330.dp, 600.dp)) {
                        Image(
                            painter = painterResource(id = proyecto.imagen),
                            contentDescription = proyecto.nombre,
                            modifier = Modifier.fillMaxSize()
                        )
                    }
                    Spacer(modifier = Modifier.height(10.dp))
                    Text(proyecto.nombre, fontSize = 16.sp, fontWeight =
                        FontWeight.Bold)
                    Text(proyecto.descripcion, fontSize = 14.sp)
                    Spacer(modifier = Modifier.height(10.dp))

                    Button(
                        // abre la Activity guardada en el proyecto
                        onClick = { context.startActivity(Intent(context,
                            proyecto.destino)) },
                        modifier = Modifier.width(150.dp).height(50.dp),
                        colors = ButtonDefaults.buttonColors(containerColor =
                            Color.DarkGray),
```

```
                        shape = RoundedCornerShape(25.dp)
                    ) {
                        Text(text = "Acceder", color = Color.White, fontWeight =
                            FontWeight.Bold)
                    }
                }
            }
        }
    }
}
```

5.4. Propinas.kt

5.4.1. Ubicación

`com.evagr.proyectitosinteractivos.proyectos.proyecto1`

Descripción general:
La clase `Propinas` es la calculadora de propinas. Guarda lo escrito en dos estados y recalcula la propina cada vez que cambian.

5.4.2. Funciones

calculateTip **Descripción:** Calcula la propina como un porcentaje de la cantidad y la devuelve con el formato de moneda del dispositivo.

Código comentado:

```
private fun calculateTip(amount: Double, tipPercent: Double): String {
    val tip = tipPercent / 100 * amount
    return NumberFormat.getCurrencyInstance().format(tip)   // por ejemplo, 1,50
}
```

NumberTextFild **Descripción:** Campo de texto reutilizable de una línea con el teclado numérico.

Código comentado:

```
@Composable
fun NumberTextFild(label: String, value: String, onValueChange: (String) -> Unit,
                   modifier: Modifier = Modifier) {
    TextField(
        label = { Text(text = label) },
        value = value,
        onValueChange = onValueChange,     // el estado vive fuera del campo
        singleLine = true,
        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
        modifier = modifier
    )
}
```

TipTimeLayout **Descripción:** Pantalla de la calculadora. Si un campo no es un número, la cantidad vale 0 y el porcentaje 15.

Código comentado:

```
@Composable
fun TipTimeLayout() {
    var inputText by remember { mutableStateOf("") }      // cantidad
    var propinaText by remember { mutableStateOf("") }    // porcentaje

    val total = inputText.toDoubleOrNull() ?: 0.0
    val porciento = propinaText.toDoubleOrNull() ?: 15.0  // 15 % por defecto
    val tip = calculateTip(total, porciento)              // se recalcula en cada
        cambio

    Column(
        modifier = Modifier.statusBarsPadding().padding(horizontal = 40.dp)
```

```
            .safeDrawingPadding().fillMaxSize(),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text(
            text = "Calculador de propinas",
            modifier = Modifier.padding(bottom = 16.dp, top = 40.dp)
                .align(alignment = Alignment.Start)
        )
        NumberTextFild(
            label = "% Porcentaje propina",
            value = propinaText,
            onValueChange = { propinaText = it },
            modifier = Modifier.padding(bottom = 32.dp).fillMaxWidth()
        )
        NumberTextFild(
            label = "Cantidad",
            value = inputText,
            onValueChange = { inputText = it },
            modifier = Modifier.padding(bottom = 32.dp).fillMaxWidth()
        )
        Text(text = "Propina de $tip", style = MaterialTheme.typography.displaySmall)
        Spacer(modifier = Modifier.height(150.dp))
    }
}
```

5.5. Botones.kt

5.5.1. Ubicación

`com.evagr.proyectitosinteractivos.proyectos.proyecto2`

Descripción general:
La clase `Botones` es el mini-proyecto de la limonada. La pantalla la dibuja la función `Greeting`, que está fuera de la clase.

5.5.2. Funciones

onCreate **Descripción:** Carga `Greeting` dentro de un `Scaffold`, que le pasa el margen de las barras del sistema.

Código comentado:

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    enableEdgeToEdge()
    setContent {
        ProyectitosInteractivosTheme {
            Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                Greeting(modifier = Modifier.padding(innerPadding))
            }
        }
    }
}
```

Greeting **Descripción:** Un estado entero (`case`) indica el paso actual, de 0 a 3. Cada paso muestra su texto y su imagen; el botón suma uno y, en el último, vuelve a 0. Se muestran el primer paso y el último; el segundo y el tercero son iguales cambiando el texto y la imagen.

Código comentado:

```
@Composable
fun Greeting(modifier: Modifier = Modifier) {
    val lemonTree = painterResource(R.drawable.lemon_tree)
    val lemonSequeeze = painterResource(R.drawable.lemon_squeeze)
    val lemonDrink = painterResource(R.drawable.lemon_drink)
    val lemonRestart = painterResource(R.drawable.lemon_restart)

```

```
    var case by remember() { mutableStateOf(0) }   // paso actual

    Column(
        modifier = Modifier.statusBarsPadding().padding(horizontal = 40.dp)
            .safeDrawingPadding().fillMaxSize(),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        if (case == 0) {
            Text(text = "Pulsa siguiente para coger un limon", modifier = modifier)
            Image(painter = lemonTree, contentDescription = "Arbol")
            Button(onClick = { case += 1 }, modifier = Modifier.padding(top = 10.dp))
                {
                Text(text = "Siguiente")
            }
        } else if (case == 1) {
            // "Pulsa siguiente para hacer una limonada" + lemonSequeeze
        } else if (case == 2) {
            // "Pulsa siguiente para beberte la limonada" + lemonDrink
        } else if (case == 3) {
            Text(text = "Pulsa siguiente para volver al arbol", modifier = modifier)
            Image(painter = lemonRestart, contentDescription = "Arbol")
            Button(onClick = { case = 0 }, modifier = Modifier.padding(top = 10.dp)) {
                Text(text = "Siguiente")       // vuelve al principio
            }
        }
    }
}
```

5.6. Login.kt

5.6.1. Ubicación

`com.evagr.proyectitosinteractivos.proyectos.proyecto3`

Descripción general:
La clase `Login` es un formulario de inicio de sesión sencillo, con el estado guardado en la propia pantalla con `remember`.

5.6.2. Funciones

MyTextField **Descripción:** Campo de texto reutilizable. Recibe una `VisualTransformation` opcional para ocultar la contraseña.

Código comentado:

```
@Composable
fun MyTextField(label: String, value: String, onValueChange: (String) -> Unit,
                modifier: Modifier = Modifier,
                visualTransformation: VisualTransformation =
                    VisualTransformation.None) {
    TextField(
        label = { Text(text = label) },
        value = value,
        onValueChange = onValueChange,
        singleLine = true,
        modifier = modifier,
        visualTransformation = visualTransformation   // None o
            PasswordVisualTransformation
    )
}
```

InputText **Descripción:** Pantalla del login: imagen de perfil, los dos campos y el botón **Entrar**, que comprueba que no haya ninguno vacío.

Código comentado:

```
@Composable
```

```
fun InputText(context: Context) {
    var inputTextUsuario by remember { mutableStateOf("") }
    var inputTextContrasena by remember { mutableStateOf("") }
    val imagenPerfil = painterResource(R.drawable.sin_usuario)

    Column(/* mismo margen y centrado que las otras pantallas */) {
        Image(painter = imagenPerfil, contentDescription = "Foto de perfil")
        Spacer(modifier = Modifier.height(20.dp))
        MyTextField(
            label = "Nombre de usuario",
            value = inputTextUsuario,
            onValueChange = { inputTextUsuario = it },
            modifier = Modifier.padding(bottom = 32.dp).fillMaxWidth()
        )
        MyTextField(
            label = "Contraseña",
            value = inputTextContrasena,
            onValueChange = { inputTextContrasena = it },
            visualTransformation = PasswordVisualTransformation(), // oculta la
                contraseña
            modifier = Modifier.padding(bottom = 32.dp).fillMaxWidth()
        )
        Button(
            onClick = {
                if (!inputTextUsuario.isEmpty() && !inputTextContrasena.isEmpty()) {
                    Toast.makeText(context, "Login correcto",
                        Toast.LENGTH_SHORT).show()
                } else {
                    Toast.makeText(context, "Rellena todos los campos",
                        Toast.LENGTH_SHORT).show()
                }
            },
            modifier = Modifier.padding(top = 10.dp)
        ) {
            Text(text = "Entrar")
        }
    }
}
```

5.7. Miniaturas.kt

5.7.1. Ubicación

`com.evagr.proyectitosinteractivos.proyectos.proyecto4`

Descripción general:
La data class `Miniaturas` guarda una tarjeta de la galería. Las anotaciones `@StringRes` y `@DrawableRes` hacen que Android Studio avise si se pasa un id que no es un texto o una imagen.

5.7.2. Funciones

Miniaturas **Descripción:** Constructor con el id del texto y el id de la imagen.
Código comentado:

```
data class Miniaturas(
    @StringRes val nombreMiniatura: Int,    // texto de strings.xml
    @DrawableRes val imagenMiniatura: Int   // imagen de drawable-nodpi
)
```

5.8. MisMiniaturas.kt

5.8.1. Ubicación

`com.evagr.proyectitosinteractivos.proyectos.proyecto4`

Descripción general:
La clase `MisMiniaturas` crea la lista de las ocho tarjetas de la galería.

5.8.2. Funciones

cargarMiniaturas **Descripción:** Función del `companion object`, así que se llama sin crear un objeto: `MisMiniaturas.cargarMiniaturas()`.

Código comentado:

```
class MisMiniaturas {
    companion object {
        fun cargarMiniaturas(): ArrayList<Miniaturas> {
            val listaAuxiliar: ArrayList<Miniaturas> = ArrayList<Miniaturas>()
            listaAuxiliar.add(Miniaturas(R.string.SuperPusheenicorn,
                R.drawable.bicho1))
            listaAuxiliar.add(Miniaturas(R.string.PusheenHeart, R.drawable.bicho2))
            // ... seis más, de bicho3 a bicho8
            return listaAuxiliar
        }
    }
}
```

5.9. Galeria.kt

5.9.1. Ubicación

`com.evagr.proyectitosinteractivos.proyectos.proyecto4`

Descripción general:
La clase `Galeria` muestra las miniaturas en una lista vertical.

5.9.2. Funciones

GaleriaApp **Descripción:** Carga las miniaturas y se las pasa a la lista.

Código comentado:

```
@Composable
fun GaleriaApp() {
    ListaMiniaturas(MisMiniaturas.cargarMiniaturas())
}
```

ListaMiniaturas **Descripción:** Un `LazyColumn` que dibuja una `MiniaturaCard` por cada miniatura.

Código comentado:

```
@Composable
fun ListaMiniaturas(listaMinis: ArrayList<Miniaturas>, modifier: Modifier = Modifier)
    {
    LazyColumn(modifier = Modifier) {
        items(listaMinis) { miniaturas ->
            MiniaturaCard(miniaturas, modifier = Modifier.padding(8.dp))
        }
    }
}
```

MiniaturaCard **Descripción:** Tarjeta con la imagen recortada a todo el ancho (`ContentScale.Crop`) y el nombre debajo.

Código comentado:

```
@Composable
fun MiniaturaCard(mini: Miniaturas, modifier: Modifier = Modifier) {
    Card(modifier = Modifier.padding(8.dp)) {
        Column(
```

```
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center
        ) {
            Image(
                painter = painterResource(mini.imagenMiniatura),
                contentDescription = stringResource(mini.nombreMiniatura),
                modifier = Modifier.fillMaxWidth(),
                contentScale = ContentScale.Crop       // rellena el ancho sin
                    deformar
            )
            Text(
                text = LocalContext.current.getString(mini.nombreMiniatura),
                modifier = Modifier.padding(16.dp),
                style = MaterialTheme.typography.headlineSmall
            )
        }
    }
}
```

5.10. Pregunta.kt

5.10.1. Ubicación

`com.evagr.proyectitosinteractivos.proyectos.proyecto5`

Descripción general:
La data class genérica `Pregunta<T>` guarda el enunciado y la respuesta correcta, que puede ser de cualquier tipo: `Boolean`, `Int`, `Double` o `String`.

5.10.2. Funciones

Pregunta **Descripción:** El tipo `T` se deduce del valor que se le pasa al crearla.
Código comentado:

```
data class Pregunta<T>(
    val pregunta: String,   // enunciado
    val respuesta: T        // respuesta correcta, del tipo que sea
)
```

5.11. MisPreguntas.kt

5.11.1. Ubicación

`com.evagr.proyectitosinteractivos.proyectos.proyecto5`

Descripción general:
La clase `MisPreguntas` crea las diez preguntas del test.

5.11.2. Funciones

cargarPreguntas **Descripción:** La lista es de tipo `Pregunta<*>` para que pueda mezclar preguntas con respuestas de tipos distintos.
Código comentado:

```
class MisPreguntas {
    companion object {
        // Pregunta es genérica: <*> indica que cada una puede ser de un tipo
        fun cargarPreguntas(): ArrayList<Pregunta<*>> {
            val listaAuxiliar: ArrayList<Pregunta<*>> = ArrayList<Pregunta<*>>()
            listaAuxiliar.add(Pregunta("¿Kotlin es orientado a objetos?
                (true/false)", true))
            listaAuxiliar.add(Pregunta("¿Cuánto es 5 + 3?", 8))
```

```
            listaAuxiliar.add(Pregunta("Palabra clave para variable inmutable",
                "val"))
            listaAuxiliar.add(Pregunta("¿Cuánto es 10.5 + 2.5?", 13.0))
            listaAuxiliar.add(Pregunta("¿Qué palabra define una función?", "fun"))
            // ... cinco más
            return listaAuxiliar
        }
    }
}
```

5.12. Score.kt

5.12.1. Ubicación

`com.evagr.proyectitosinteractivos.proyectos.proyecto5`

Descripción general:
`Score` es un `object`: solo existe una instancia en toda la app y guarda los aciertos del test.

5.12.2. Funciones

score **Descripción:** Contador de aciertos; se pone a 0 antes de corregir.
Código comentado:

```
object Score {
    var score: Int = 0
}
```

5.13. Formulario.kt

5.13.1. Ubicación

`com.evagr.proyectitosinteractivos.proyectos.proyecto5`

Descripción general:
La clase `Formulario` es el test. Guarda lo que escribe el usuario en el array `respuestas` y lo compara con las respuestas correctas al pulsar **Finalizar Test**.

5.13.2. Funciones

ListaPreguntas **Descripción:** Una columna con la lista de preguntas, que ocupa todo el espacio libre (`weight(1f)`), y el botón fijo abajo.
Código comentado:

```
var respuestas = Array(10) { "" }      // una respuesta por pregunta

@Composable
fun ListaPreguntas(listaPreg: ArrayList<Pregunta<*>>, modifier: Modifier = Modifier) {
    val contexto = LocalContext.current
    Column(modifier = Modifier.fillMaxSize()) {
        LazyColumn(modifier = Modifier.weight(1f)) {   // el botón queda siempre
            visible
            item { PreguntaCard(listaPreg[0], 0) }
            item { PreguntaCard(listaPreg[1], 1) }
            // ... hasta la 9
        }
        Button(
            onClick = {
                Score.score = 0                    // se reinicia antes de corregir
                validarRespuestas(listaPreg)
                Toast.makeText(contexto, "Has acertado ${Score.score} preguntas",
                    Toast.LENGTH_LONG).show()
            },
```

```
            modifier = Modifier.fillMaxWidth().padding(16.dp)
        ) {
            Text("Finalizar Test")
        }
    }
}
```

PreguntaCard **Descripción:** Tarjeta con el enunciado y un campo de texto. Lo escrito se guarda en el estado de la tarjeta y también en `respuestas[index]`.

Código comentado:

```
@Composable
fun PreguntaCard(preg: Pregunta<*>, index: Int, modifier: Modifier = Modifier) {
    var inputText by remember { mutableStateOf(respuestas[index]) }
    Card(modifier = Modifier.padding(8.dp)) {
        Column(
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center
        ) {
            Text(text = preg.pregunta, modifier = Modifier.padding(16.dp),
                style = MaterialTheme.typography.headlineSmall)
            TextField(
                label = { Text("Escribe tu respuesta") },
                value = inputText,
                onValueChange = {
                    inputText = it
                    respuestas[index] = it          // se guarda para corregir después
                },
                modifier = Modifier.fillMaxWidth().padding(16.dp)
            )
        }
    }
}
```

validarRespuestas **Descripción:** Compara cada respuesta con la correcta según su tipo: los números se convierten con `toIntOrNull` o `toDoubleOrNull` y el resto se compara como texto. Cada acierto suma uno a `Score.score`.

Código comentado:

```
fun validarRespuestas(listaPreg: ArrayList<Pregunta<*>>) {
    for (i in 0 until listaPreg.size) {
        val user = respuestas[i]
        val correcta = listaPreg[i].respuesta
        // boolean: se compara el texto "true"
        if (i == 0 && user == correcta.toString()) { Score.score++ }
        // int: si no es un número, toIntOrNull devuelve null y no acierta
        if (i == 1 && user.toIntOrNull() == correcta) { Score.score++ }
        // texto
        if (i == 2 && user == correcta.toString()) { Score.score++ }
        // double
        if (i == 3 && user.toDoubleOrNull() == correcta) { Score.score++ }
        // ... igual para las preguntas 4 a 9
    }
}
```

5.14. UIState.kt

5.14.1. Ubicación

`com.evagr.proyectitosinteractivos.proyectos.proyecto6`

Descripción general:
La data class `UIState` reúne todo el estado de la pantalla del login avanzado.

5.14.2. Funciones

UIState **Descripción:** Nombre y contraseña, vacíos por defecto.

Código comentado:

```
data class UIState(
    val nombre: String = "",
    val contraseña: String = "",
)
```

5.15. MainViewModel.kt

5.15.1. Ubicación

`com.evagr.proyectitosinteractivos.proyectos.proyecto6`

Descripción general:
`MainViewModel` guarda el estado del login en un `MutableStateFlow` privado y lo expone hacia fuera como un `StateFlow` de solo lectura. Así, la pantalla solo puede cambiarlo con las funciones del ViewModel. En el archivo quedan comentadas dos versiones anteriores (con `mutableStateOf` y con un contador) que sirvieron para practicar.

5.15.2. Funciones

uiState **Descripción:** Estado de solo lectura que observa la pantalla.
Código comentado:

```
private val _uiState = MutableStateFlow(UIState())   // solo se modifica aquí dentro
val uiState: StateFlow<UIState> = _uiState            // la vista solo lo lee
```

cambiarNombre **Descripción:** Crea una copia del estado con el nombre nuevo.
Código comentado:

```
fun cambiarNombre(nuevoNombre: String) {
    _uiState.update { currentState ->
        currentState.copy(nombre = nuevoNombre)   // copy: el resto no cambia
    }
}
```

cambiarContraseña **Descripción:** Igual que la anterior, con la contraseña.
Código comentado:

```
fun cambiarContraseña(nuevaContraseña: String) {
    _uiState.update { currentState ->
        currentState.copy(contraseña = nuevaContraseña)
    }
}
```

5.16. MainActivity.kt

5.16.1. Ubicación

`com.evagr.proyectitosinteractivos.proyectos.proyecto6`

Descripción general:
`MainActivity` es el login con ViewModel. Crea el `MainViewModel` y se lo pasa a la función `LoginApp`, que dibuja la pantalla.

5.16.2. Funciones

onCreate **Descripción:** Carga `LoginApp` con el ViewModel de la Activity.
Código comentado:

```kotlin
class MainActivity : ComponentActivity() {
    val viewModelTest = MainViewModel()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            ProyectitosInteractivosTheme {
                LoginApp(viewModelTest)
            }
        }
    }
}
```

LoginApp **Descripción:** Lee el estado con `collectAsState()`, de forma que la pantalla se vuelve a dibujar cada vez que cambia. Los campos no guardan nada: avisan al ViewModel.

Código comentado:

```kotlin
@Composable
fun LoginApp(viewModel: MainViewModel = MainViewModel()) {
    val uiState by viewModel.uiState.collectAsState()   // se recompone con cada cambio
    val imagenPerfil = painterResource(R.drawable.sin_usuario)
    val context = LocalContext.current

    Column(/* mismo margen y centrado que las otras pantallas */) {
        Image(painter = imagenPerfil, contentDescription = "Foto de perfil",
            modifier = Modifier.height(80.dp))
        Spacer(modifier = Modifier.height(100.dp))
        TextField(
            value = uiState.nombre,
            onValueChange = { viewModel.cambiarNombre(it) },   // el estado está en el ViewModel
            label = { Text(text = "Introduce tu nombre") },
            modifier = Modifier.padding(bottom = 32.dp).fillMaxWidth()
        )
        TextField(
            value = uiState.contraseña,
            onValueChange = { viewModel.cambiarContraseña(it) },
            label = { Text(text = "Introduce tu contraseña") },
            visualTransformation = PasswordVisualTransformation(),
            modifier = Modifier.padding(bottom = 32.dp).fillMaxWidth()
        )
        Button(
            onClick = {
                if (!uiState.nombre.isEmpty() && !uiState.contraseña.isEmpty()) {
                    Toast.makeText(context, "Login correcto, \"Hola, ${uiState.nombre}\"",
                        Toast.LENGTH_SHORT).show()
                } else {
                    Toast.makeText(context, "Rellena todos los campos",
                        Toast.LENGTH_SHORT).show()
                }
            },
            modifier = Modifier.padding(top = 10.dp)
        ) {
            Text(text = "Entrar")
        }
    }
}
```

5.17. Theme.kt, Color.kt y Type.kt

5.17.1. Ubicación

`com.evagr.proyectitosinteractivos.ui.theme`

Descripción general:
Son los archivos del tema que crea Android Studio al hacer un proyecto de Compose: `ProyectitosInteractivos`

aplica los colores de Material 3 (claros u oscuros según el sistema, y los colores dinámicos en Android 12 o superior) y la tipografía. Todas las pantallas de la app se dibujan dentro de este tema.

5.17.2. Funciones

6. Mejoras futuras

- Terminar los **ajustes visuales** que quedaron pendientes y dar a todos los mini-proyectos el mismo estilo, con un título y un botón para volver al menú.
- Pasar de una `Activity` por mini-proyecto a una sola con **Navigation Compose**.
- Crear el ViewModel con `by viewModels()` para que el login avanzado no pierda lo escrito al girar el móvil.
- Llevar el estado de más mini-proyectos a un `ViewModel`, como en el login avanzado.
- Corregir el test sin depender de la posición de cada pregunta, comprobando el tipo de la respuesta con `when`, y aceptar `Double` con mayúscula en la pregunta del tipo de dato.
- Poner tildes y descripciones distintas en las imágenes de la limonada (ahora todas son *«Arbol»*).
- Usar imágenes propias en la galería.
- Añadir **tests** de la lógica de las propinas y de la corrección del test.
- Mover `Home` del paquete `ui.theme` a uno propio.